Backlog Granular de Ejecución (Dinámico y Exhaustivo)

Este documento contiene la hoja de ruta de ejecución atómica EXTENSA para la plataforma **Templados AL13**. Cada Épica está desarrollada hasta su nivel más bajo operable por IAs.

🧬 Protocolo de Evolución Dinámica (The Mutation Protocol)

Las tareas marcadas con `[ ]` están pendientes. Las nuevas subtareas orgánicas (bugs/cambios) se inyectarán debajo del bloque afectado usando prefijos:

- `[FIX]` : Resolución de bugs (Ej. Conflictos de dependencias).
- `[REF]` : Refactorización de código legado o ineficiente.
- `[ADD]` : Nuevo requerimiento atómico (Feature Request).

🏗️ Epic 1: Scaffolding y Arquitectura Base (Cimientos)

Objetivo: Establecer el entorno de desarrollo, linters, CI/CD y conexión a base de datos segura.

- **Task 1.1: Inicializar el Monolito Front-end (Astro + Svelte 5)**
  - [ ] 1.1.1: Ejecutar `npm create astro@latest` (template `basics`, estricto, sin UI, TypeScript `strict`).
  - [ ] 1.1.2: Inicializar repositorio Git y añadir `.gitignore` estándar de Node/Astro/Entorno.
  - [ ] 1.1.3: Limpiar archivos basura generados por defecto (CSS global, SVG de muestra en `public/`).
  - [ ] 1.1.4: Ejecutar `npx astro add svelte` para integrar el motor reactivo de UI.
  - [ ] 1.1.5: Configurar `astro.config.mjs` para asegurar compatibilidad de Svelte 5 (Runes).
  - [ ] 1.1.6: Crear estructura base: `src/components/ui/`, `src/layouts/`, `src/lib/utils/`, `src/assets/`.
  - [ ] 1.1.7: Escribir test unitario básico (Vitest) para asegurar el montaje Astro-Svelte.
  - [ ] 1.1.8: Commit base de andamiaje ("chore: init astro svelte workspace strict mode").
- **Task 1.2: Configurar Sistema UI y Tailwind CSS (Glassmorphism)**
  - [ ] 1.2.1: Ejecutar `npx astro add tailwind` o configuración manual de PostCSS.
  - [ ] 1.2.2: Modificar `tailwind.config.mjs` incrustando paleta Hex AL13 (Oscuros, Dorados/Plateados industriales).
  - [ ] 1.2.3: Definir en Tailwind opacidades personalizadas para desenfoques `backdrop-blur`.
  - [ ] 1.2.4: Instalar y configurar web fonts (Inter para lectura, Outfit para Headers) vía `@fontsource`.
  - [ ] 1.2.5: Crear `src/styles/global.css` implementando utilitarios puros pre-compilados (`.glass-panel`, `.text-gradient`).
  - [ ] 1.2.6: Componentizar un `Badge.svelte` de prueba aplicando Glassmorphism para validar diseño.
- **Task 1.3: Setup de Repositorio de Datos (Supabase/PostgreSQL)**
  - [ ] 1.3.1: Instalar Supabase CLI v1 y ejecutar `supabase init` localmente.
  - [ ] 1.3.2: Generar y configurar archivos `.env.example` y obligar `.env` en Gitignore.
  - [ ] 1.3.3: Diseñar e implementar DDL (migration SQL) `001_initial_schema.sql` (Usuarios, Roles, Perfil AL13).

- [ ] 1.3.4: Diseñar DDL (migration SQL) `002_leads_schema.sql` (Cotizaciones, Status Kanban, Valores $).
- [ ] 1.3.5: Levantar motor local `supabase start` y validar conexión pgAdmin/Studio.
- [ ] 1.3.6: Generar tipos TypeScript estáticos a partir de Supabase (`supabase gen types typescript --local > src/lib/database.types.ts`).

- **Task 1.4: Protocolos DevOps y Calidad (CI/CD)**
  - [ ] 1.4.1: Instalar ESLint y Prettier Plugins para Astro y Svelte.
  - [ ] 1.4.2: Configurar `.prettierrc` (Tabulaciones, comillas simples, line width).
  - [ ] 1.4.3: Instalar `husky` y `lint-staged` para ganchos `pre-commit` automáticos.
  - [ ] 1.4.4: Escribir GitHub Action `.github/workflows/ci.yml` para correr Linting + TypeCheck en cada PR.

🎨 Epic 2: The Visual Hero & Landing Page B2B/B2C (First Impression)

Objetivo: Crear un escaparate público cinemático que asegure tiempos de carga hiper-rápidos (SEO Edge).

- **Task 2.1: Layout Maestro Público**
  - [ ] 2.1.1: Crear componente envolvente `PublicLayout.astro` manejando `<head>`, meta tags y SEO Title/Description dinámicos.
  - [ ] 2.1.2: Implementar componente genérico Header/Nav (`Navbar.svelte`) con logo AL13 y scroll-spy (cambia color a transparente).
  - [ ] 2.1.3: Implementar componente genérico Footer estático (Enlaces, Términos Legales, Redes).

- **Task 2.2: Componente "Hero Section" Premium**
  - [ ] 2.2.1: Escribir tests unitarios Vitest definiendo existencia de textos principales y CTA de botón.
  - [ ] 2.2.2: Construir HTML semántico de `Hero.svelte` (Título h1 impactante y Párrafo "Boutique Industrial").
  - [ ] 2.2.3: Aplicar clases CSS `glass-panel` creadas en la Epic 1.2.
  - [ ] 2.2.4: Refinar el botón CTA OnHover: Transición de 300ms a `scale(1.02)` y brillo interior sutil (Shadow).
  - [ ] 2.2.5: Injectar un video de fondo comprimido (o imagen oscura abstracta) con overlay oscuro para contraste del texto.

- **Task 2.3: Value Proposition (Grid de Beneficios B2B / B2C)**
  - [ ] 2.3.1: Crear componente aislable `FeatureCard.svelte` (Ícono, Título, Descripción corta).
  - [ ] 2.3.2: Crear el grid iterador `FeaturesGrid.astro` que lea datos (arrays) mockeados.
  - [ ] 2.3.3: Integrar animaciones de entrada AOS (Animate on Scroll) a las tarjetas para suavidad visual.

- **Task 2.4: Optimización Image Delivery y Web Vitals**
  - [ ] 2.4.1: Convertir de lote todos los `png/jpg` a `webp/avif` usando componente nativo `<Image />` de Astro.
  - [ ] 2.4.2: Asegurar carga diferida `loading="lazy"` en todas las imágenes por debajo del "fold" (del inicio de pantalla).

- [ ] 2.4.3: Revisar métricas Lighthouse CLI locales buscando puntuación perfecta en Performance/Accessibilidad.

🧮 Epic 3: La Calculadora B2B Determinista (Core Logic)

Objetivo: Permitir cotizaciones en vivo (State Machine Svelte) y captura asíncrona de Leades ($5M a 3 min).

- **Task 3.1: Backend - Endpoint de Ingestión de Cotización (Leads)**
  - [ ] 3.1.1: Crear archivo de API nativa en Astro `src/pages/api/leads.ts` (`POST`).
  - [ ] 3.1.2: Validar Payload JSON de entrada usando librería `Zod` (Medidas, Nombre, Empresa, Teléfono).
  - [ ] 3.1.3: Cifrar datos o prevenir Inyecciones SQL usando el ORM/Supabase SDK.
  - [ ] 3.1.4: Grabar el `Lead` en la BD Supabase y retornar Status 201 (Created) con Lead_ID.
  - [ ] 3.1.5: Escribir test de integración enviando POST falso al endpoint y verificando validación Zod.
- **Task 3.2: Frontend - UI Svelte State Machine (The Quoter Engine)**
  - [ ] 3.2.1: Definir `store` global temporal o Runes (`$state`) para almacenar las selecciones del usuario en el navegador (Paso a Paso).
  - [ ] 3.2.2: Construir UI Paso 1: Selección visual del tipo de producto (Puerta de Baño, Ventanal, etc).
  - [ ] 3.2.3: Construir UI Paso 2: Input de Medidas (Alto x Ancho) con validadores rígidos (ej. máximo 2200mm).
  - [ ] 3.2.4: Construir UI Paso 3: Tipos de Vidrio y Perfilería (10mm vs 8mm).
  - [ ] 3.2.5: Construir UI Paso 4: Formulario final de datos del Contractor / B2C para "Enviar y Ver Precio".
- **Task 3.3: Motor Matemático (Precios y Tolerancias)**
  - [ ] 3.3.1: Aislar la clase/lógica JS en `src/lib/calculusEngine.ts`.
  - [ ] 3.3.2: Escribir tests unitarios que verifiquen el precio del m2 contra la matriz base.
  - [ ] 3.3.3: Implementar condición: Si Alto > 2100mm = Advertencia Viento / Forzar vidrio 10mm.
  - [ ] 3.3.4: Conectar retorno del JS lógico a la vista final del resumen de la cotización en Svelte.
- **Task 3.4: Notificaciones Asíncronas Push (El CEO se entera)**
  - [ ] 3.4.1: Integrar Resend / SendGrid SDK dentro del endpoint `/api/leads.ts`.
  - [ ] 3.4.2: Diseñar plantilla HTML de correo ("Nuevo Lead B2B de $XX COP").
  - [ ] 3.4.3: Testear envío asíncrono hacia correo corporativo del CEO AL13 (sin bloquear el loading del UI del usuario final).

🛡️ Epic 4: El Panel de Control Admin (CMS Privado Soberano)

Objetivo: Interfaz secreta y altamente segura para gestión comercial, Kanban y cambios de catálogo.

- **Task 4.1: Bouncer de Seguridad y Auth Routing**
  - [ ] 4.1.1: Crear componente `/admin/login` con formulario de acceso único (Supabase Auth Email/Password).

- 4.1.2: Implementar SSR Layout `AdminLayout.astro` que intercepte Cookies JWT de Supabase en cada request.
- 4.1.3: Redireccionar forzosamente a `/admin/login` si la Cookie de Sesión está expirada, previniendo acceso estático o cliente.

- **Task 4.2: CRM Dashboard B2B (Módulo Kanban de Leads)**
  - 4.2.1: Crear ruta SSR `/admin/leads` que haga fetch de todos los Leads Ordenados (`ORDER BY created_at DESC`).
  - 4.2.2: Construir componente Svelte `KanbanBoard` compuesto por columnas "Nuevo", "Cotizado", "Cerrado Ganado", "Perdido".
  - 4.2.3: Implementar librería de Drag-and-Drop (DND) ligera vinculada a Runes en Svelte.
  - 4.2.4: Crear endpoint interno (PATCH) en Astro para mutar el status de un Lead en la Base de Datos al soltar la tarjeta de arrastre (Drop).

- **Task 4.3: CMS de Catálogo (Uploader a Buckets S3/Supabase Storage)**
  - 4.3.1: Crear tabla/esquema en Base Datos para "Catálogo de Productos".
  - 4.3.2: Construir UI en Svelte: Formulario "Añadir Proyecto Exitoso".
  - 4.3.3: Programar función de subida presigned de imágenes (Imágenes de baño/cocina) directo al bucket optimizado ignorando NodeJS proxy por velocidad.
  - 4.3.4: Revalidar Caché SSG de Astro programáticamente si una imagen pública es actualizada.

🔮 Epic 5: The Render Engine (Interacción WebGL B2C 3D)

Objetivo: Funcionalidad Eye-Candy que destruya a la competencia visual, donde el usuario final pueda rotar un vidrio templado 3D en su móvil.

- **Task 5.1: Setup Core 3D (Threlte Engine)**
  - 5.1.1: Instalar la dependencia oficial `three` y `@threlte/core` compatible con Svelte 5.
  - 5.1.2: Montar componente de Canvas maestro (`3DCanvas.svelte`) asegurando z-index debajo del texto UI pero arriba del fondo.

- **Task 5.2: Ingesta de Escenarios PBR (Modelos y Materiales)**
  - 5.2.1: Optimizar modelo crudo de aluminio `.glb` mediante `gltf-pipeline` o Draco compression en terminal (Reducción MBs).
  - 5.2.2: Cargar el modelo en Threlte asíncronamente (Suspense Loader UI).
  - 5.2.3: Programar shaders fotorrealistas (MeshPhysicalMaterial) asignando "Transmission = 1.0", "Roughness < 0.1", e IOR (Índice de refracción) equivalente a vidrio clásico para el target 3D.
  - 5.2.4: Asignar coloración "Anodizado/Mate" a las estructuras del perfil perimetral metálico en ThreeJS.

- **Task 5.3: Coreografías de Cámara (Orbiting)**
  - 5.3.1: Inyectar `<OrbitControls />` limitando el pitch polar a rangos normales (el usuario no puede ver la ducha "desde abajo" u horizontes absurdos).

- 5.3.2: Auto-rotación sutil y lenta (`autoRotate={true} autoRotateSpeed={0.5}`) si el usuario no interactúa en 5 segundos.

- **Task 5.4: Degradación SRE (Site Reliability Engineering / Fallback)**
  - 5.4.1: Programar detector (script muy veloz) de capabilities WebGL en el navegador del cliente antes del montaje.
  - 5.4.2: Destruir el Canvas 3D e inyectar una "Imagen 2D de Alta Calidad (AVIF)" inmediatamente si el teléfono es obsoleto o carece de Aceleración de Hardware.

📈 Epic 6: Scalado Programático Orgánico y Telemetría (Growth)

Objetivo: Expansión hiper-escalable del sitio sin intervención humana a todos los barrios y ciudades, soportado por data real.

- **Task 6.1: Motor Generador Programático de Rutas SSR (Local SEO)**
  - 6.1.1: Crear tabla o array maestro de variables SEO (Ej. ["Bogotá Chico", "Medellín Poblado", "Puertas Plegables", "Mamparas Vidrio"]).
  - 6.1.2: Programar la plantilla dinámica Astro dinámica `/catalogo/[tipo]/[ubicacion].astro`.
  - 6.1.3: Proveer la función `getStaticPaths()` si SSG, o lectura de Base Datos SSR para inyectar miles de combinaciones en un build, dominando búsquedas nicho como "mamparas vidrio bogota chico" orgánicamente.
  - 6.1.4: Inyectar el `<Title>` y meta descripciones `h1` atómicamente relevantes para ese nicho preciso generado.

- **Task 6.2: Dashboard Telemetría Comercial Interno**
  - 6.2.1: En el módulo Admin de la Epic 4, crear endpoint estadístico global (Grafana primitivo embebido).
  - 6.2.2: Fetch agregado de "Conversion Rates" (Cuantos iniciaron la calculadora web VS cuántos dejaron sus datos).
  - 6.2.3: Gráfico de barras o ChartJS mostrando volumen monetario ($) Lead por tipo de producto semanalmente.
  - 6.2.4: Registro de logs de errores del Front-end enviados directamente a Supabase Errors Tab (Frontend Observability).